

University ERP System

Sandeep Kumar

2024504

Prem Gupta

2024435

1 Project Overview

The University ERP (Enterprise Resource Planning) System is a desktop-based application designed to manage the core academic processes of a university. The project is built using **Java Swing** for the user interface and **MySQL** for the database backend.

The primary goal of the system is to provide a streamlined platform for three distinct user roles:

- **Admins:** Responsible for managing users (students and professors), creating courses, and maintaining system status.
- **Professors:** Responsible for managing their assigned courses and grading students.
- **Students:** Able to register for courses, drop them, and view their academic grades.

The system places a strong emphasis on security by separating authentication data from university operational data and ensuring that user passwords are consistently hashed before storage.

2 Technology Stack and Architecture

2.1 Tech Stack

The project utilizes the following technologies to ensure stability and security:

- **Language:** Java (JDK 17+)
- **User Interface:** Java Swing (JFrame, JPanel for GUI components)
- **Database:** MySQL (Relational Database Management System)
- **Security:** jBCrypt (For secure password hashing)
- **Database Access:** JDBC (Java Database Connectivity)

2.2 System Architecture

The application follows a standard layered architecture, separating concerns between the user interface, business logic, and data access.

1. **UI Layer (View):** This layer handles all user interactions. It consists of dashboard classes such as `Admind.java`, `Studentd.java`, `Instructord.java`, and the entry point `LoginWindow.java`.
2. **Service Layer (Logic):** This layer contains the core business logic and acts as a bridge between the UI and the database. It includes `AdminService`, `StudentService`, `InstructorService`, and `AuthService`.
3. **Database Layer (Data):** Managed by the `Connectdb` class, this layer handles connection pooling and raw SQL execution.

3 Database Design

A key architectural decision in this project was to implement a **Dual-Database Strategy**. This separates sensitive login credentials from general university data, adhering to security best practices.

3.1 Authentication Database (authenticationdb)

This database is dedicated solely to user identity. It stores credentials securely and is accessed only during the login process.

Table	Key Columns	Function
users_auth	user_id, username	Unique user identification
	password_hash	Stores the BCrypt hash of the password
	role	Defines access level (admin, student, professor)

Table 1: Authentication Schema

3.2 University Database (universitydb)

This database stores the operational data required for the ERP's daily functions.

Table	Key Columns	Function
users	id, username, role	Profile data for students and professors
courses	id, code, credits	Course catalog and professor assignments
registrations	student_id, course_id	Links students to courses
grades	marks, letter_grade	Stores academic performance data

Table 2: Operational Schema

Data integrity is enforced strictly using Foreign Keys (linking grades to registrations, and registrations to users/courses) and Unique Constraints (preventing duplicate course codes or double registrations).

4 Key Features and Implementation Logic

4.1 Security and Authentication

Security is enforced through the AuthService. When a user attempts to log in, the system retrieves the stored hash from the users_auth table and uses the BCrypt.checkpw() method to verify the input. This ensures that plain text passwords are never exposed, even in memory. Once verified, the user is redirected to the specific dashboard associated with their role.

4.2 Role Enforcement

The system uses strict Role-Based Access Control (RBAC). The application flow splits immediately after login. An Admin is directed to the Admin Dashboard, while Students

and Professors are sent to their respective views. This is not just a UI trick; the underlying services are also distinct. For instance, the `StudentService` does not contain any methods to create courses, making it impossible for a student to trigger administrative actions.

4.3 Maintenance Mode

The Admin Dashboard includes a toggle for "Maintenance Mode." This feature allows the administrator to flip a system-wide flag in the `AdminService`. While currently visual, this flag is designed to act as a gatekeeper, preventing non-admin users from performing write operations when the system is undergoing updates.

4.4 Flexible Grading System

The grading logic is designed to be flexible rather than rigid. The system separates **Marks** (0-100) from **Letter Grades** (A, B, etc.).

Instead of automatically calculating a grade based on a fixed formula, the Professor has the discretion to assign marks and letter grades independently. This allows for curving or adjustments based on class performance. Technically, this is handled using an "Upsert" strategy (Update or Insert), which allows professors to update a student's grade multiple times without causing database errors or creating duplicate records.

Screenshots

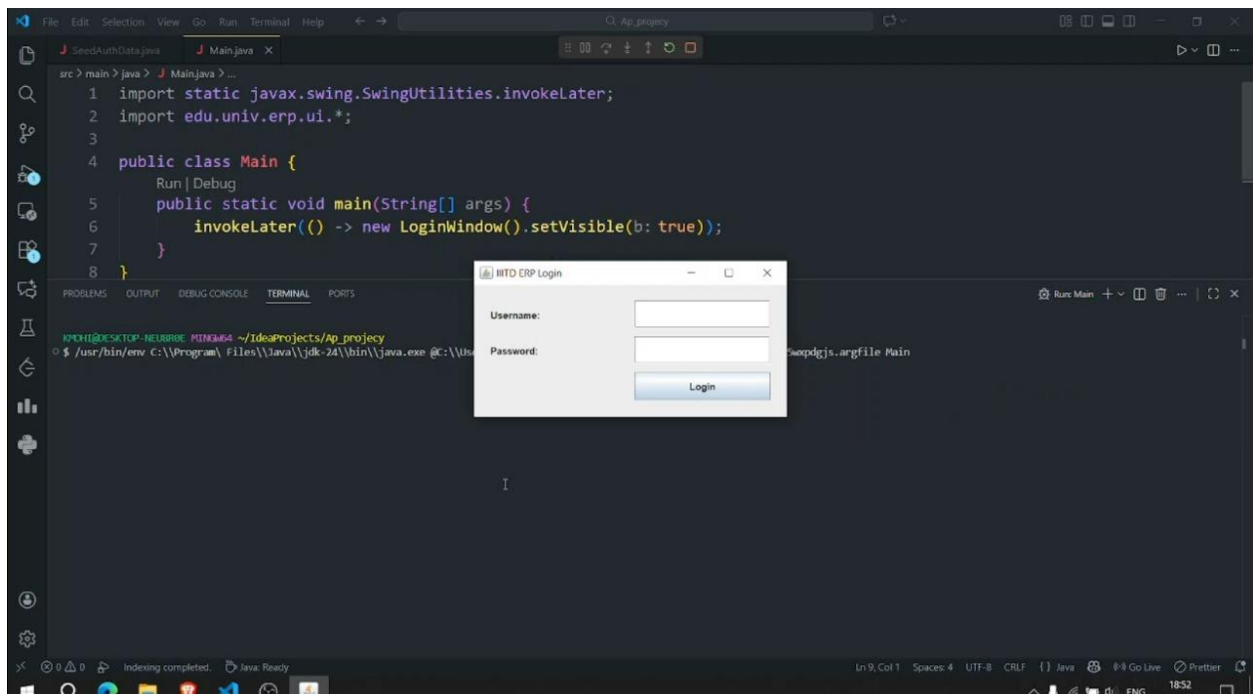


Figure 1: Login Window: Secure entry point for all users.

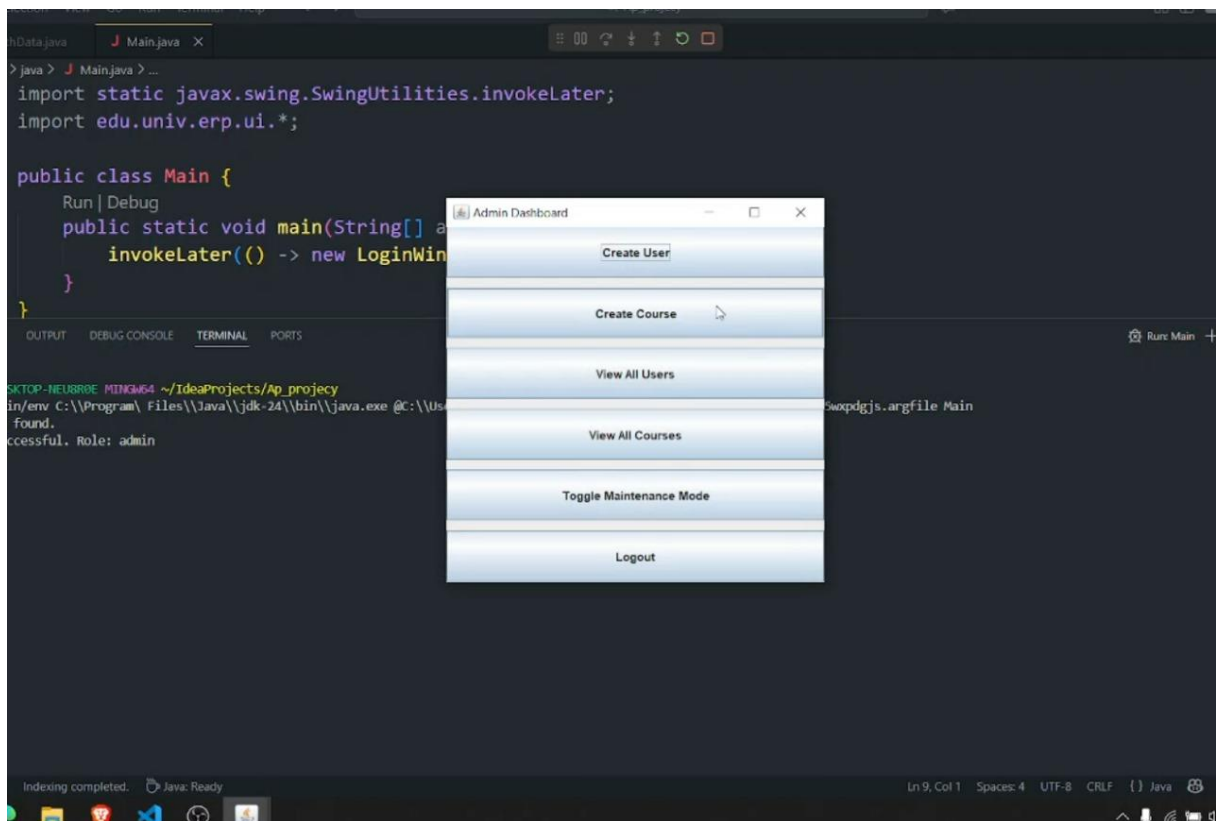


Figure 2: Admin Dashboard: Central hub for managing users and courses.

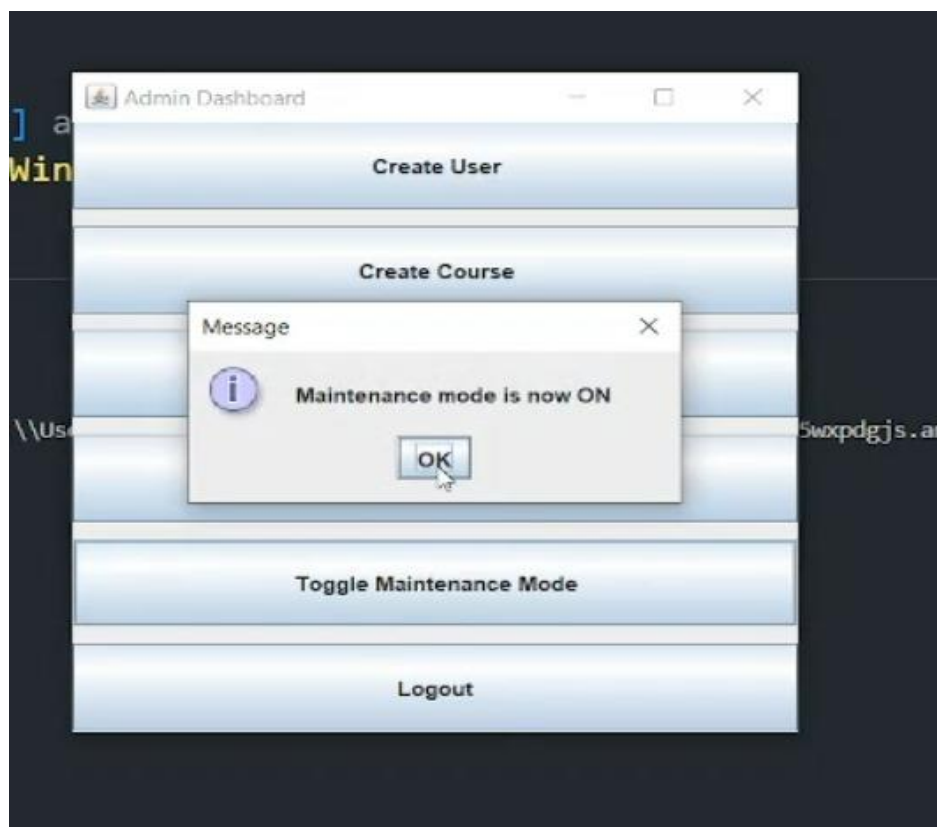


Figure 3: Maintenance Mode: Admin alerting the system state change.

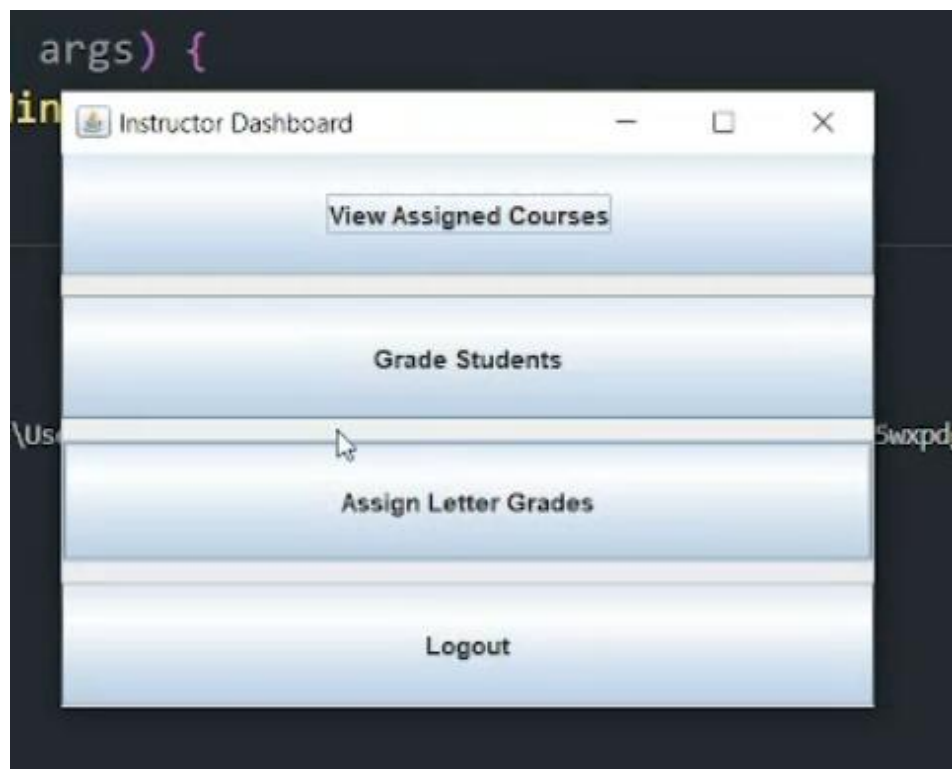


Figure 4: Instructor Dashboard: Interface for grading and course views.

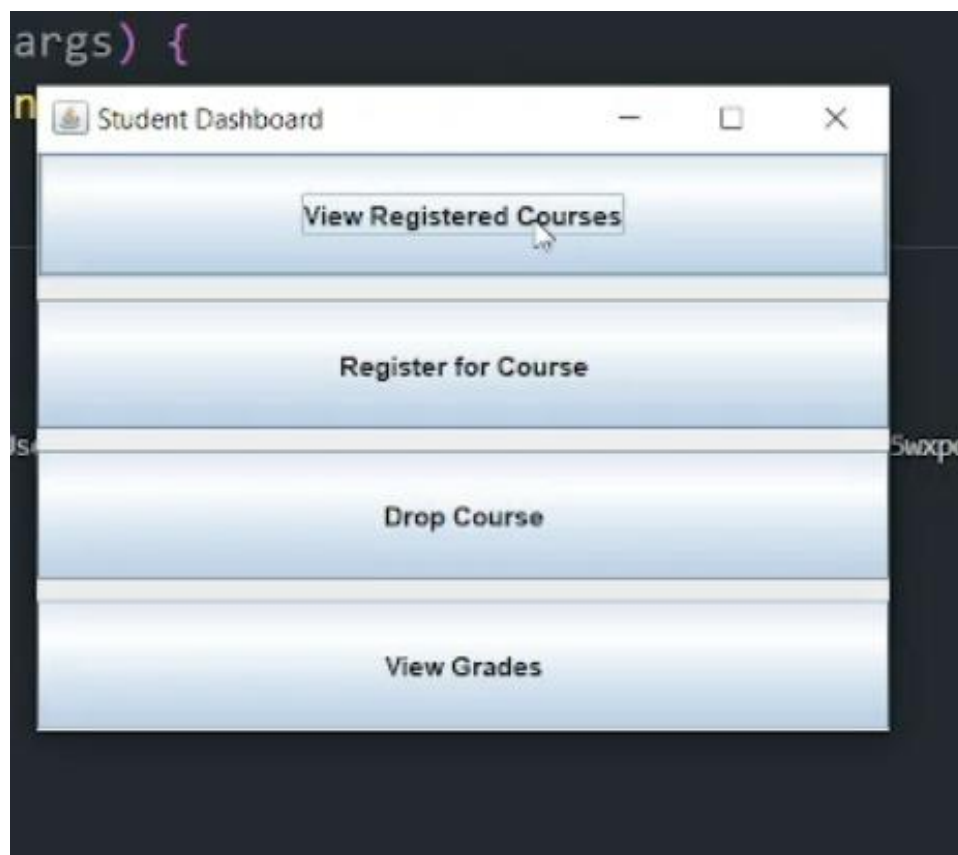


Figure 5: Student Dashboard: Interface for course registration and dropping.